

YS Admin (CMS) — Comprehensive Architecture, Operations & Security Manual

System Version: 2.4.0-Hardened

Environment: Next.js 16 (App Router) / React 19 / Prisma 7 / PostgreSQL (Supabase) / NextAuth.js

Author: Engineering Team

Classification: Internal Engineering & Operational Documentation

Table of Contents

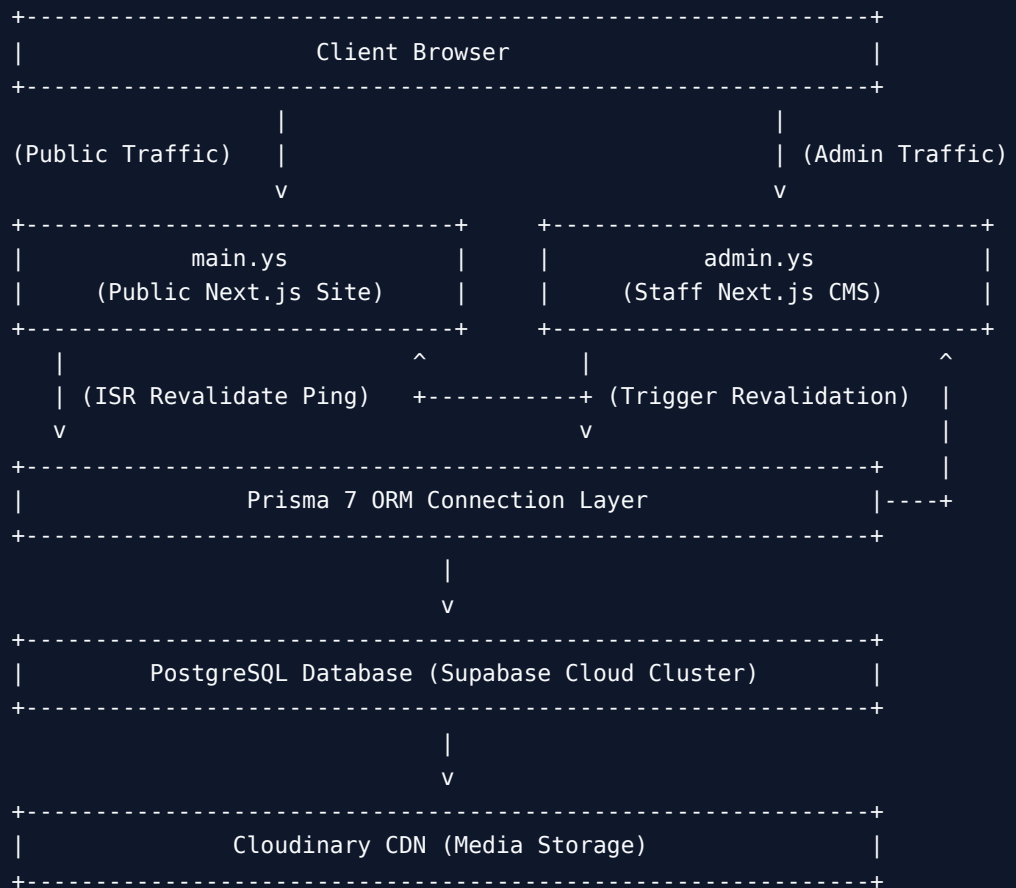
1. [System Overview & Architecture Topology](#)
2. [Role-Based Access Control \(RBAC\) Matrix](#)
3. [PART 1: Operational User Manual \(Action-Driven UI & System Workflows\)](#)
 - [Module 1: Authentication & Session Lifecycle](#)
 - [Module 2: Dashboard Overview & Telemetry](#)
 - [Module 3: Webpages Management](#)
 - [Module 4: Live Visual Block Editor](#)
 - [Module 5: Blog Publishing Engine](#)
 - [Module 6: Comments & Discussions Moderation](#)
 - [Module 7: Global Header Navigation Manager](#)
 - [Module 8: Global Footer & Legal Manager](#)
 - [Module 9: Cloudinary Media Library](#)
 - [Module 10: Staff & User Management](#)
 - [Module 11: Notification Center](#)
 - [Module 12: Redirects & Canonical Routing](#)
 - [Module 13: "Aw, Snap!" Tab Crash Prank Screen](#)
4. [PART 2: Deep-Dive Security & Hardening Architecture](#)
 - [2.1 Cryptographic Password Storage \(Bcrypt 12 Salt Rounds\)](#)
 - [2.2 Brute-Force Defense & In-Memory Rate Limiting](#)
 - [2.3 Anti-Bot Honeypot Defense Architecture](#)
 - [2.4 Identity Spoofing Mitigation \(Authoritative Badging\)](#)
 - [2.5 Object-Level Authorization & RBAC Enforcement](#)
 - [2.6 Timing-Safe ISR Cache Purging](#)
 - [2.7 HTTP Hardening & Network Transport Security](#)

5. PART 3: Architectural Roadmap & Future RFCs

- [RFC 1: Real-Time Team Presence & Collaborative Page Locking](#)
- [RFC 2: Decoupled Python SEO Audit & Crawling Engine](#)
- [RFC 3: Enterprise Immutable Audit Logging](#)

1. System Overview & Architecture Topology

The **YS Admin CMS** is a headless content and configuration engine powering the public web client (`main.js`). The system is architected around decoupling content entry from public consumption to guarantee high performance, security isolation, and instantaneous page delivery.



Key Technical Characteristics:

1. **Frontend CMS Engine:** Next.js 16 with React 19 compiler optimization and Server Actions.
2. **Database & ORM:** PostgreSQL running on Supabase, abstracted via Prisma 7 with pooled direct connections.
3. **Media CDN:** Cloudinary unsigned presets for client-side direct uploads, bypassing application memory overhead.

4. **Cache Invalidation:** On-demand Incremental Static Regeneration (ISR) using HMAC-secured timing-safe revalidation webhooks.

2. Role-Based Access Control (RBAC) Matrix

The system enforces strict multi-tier role authorization. Privileges are verified at both the middleware layer (`proxy.ts`) and individual API route handlers.

Capability / Resource	<code>ADMIN</code> Role	<code>EDITOR</code> Role	Unauthenticated Visitor
Access CMS Dashboard	Granted	Granted	Redirected to <code>/login</code>
Create / Edit Own Blog Posts	Granted	Granted	Denied (401)
Edit / Delete Other Authors' Blogs	Granted	Denied (403)	Denied (401)
Create Web Pages	Granted	Granted	Denied (401)
Delete Web Pages	Granted	Denied (403)	Denied (401)
Cloudinary Media Upload	Granted	Granted	Denied (401)
Permanently Destroy Media	Granted	Denied (403)	Denied (401)
Create Staff Users & Set Roles	Granted	Denied (403)	Denied (401)
Moderate & Reply to Comments	Granted (<code>isAdmin: true</code>)	Granted (<code>isAdmin: true</code>)	Public form only
Global Header / Footer Config	Granted	Granted	Read-only public

PART 1: Operational User Manual

(Action-Driven UI & System Workflows)

This section documents every administrative capability in the exact format requested by engineering leads:
Action $\rightarrow$ **UI Trigger** $\rightarrow$ **System Execution** $\rightarrow$ **Database Mutations** $\rightarrow$ **Cache Effects** $\rightarrow$ **Edge Case Failures**.

Module 1: Authentication & Session Lifecycle

- **Path:** `/login`

- **Target Audience:** All staff members (`ADMIN` and `EDITOR`).

Action 1.1: Staff Sign-In

- **Trigger:** User fills in Email + Password and clicks "Sign In".
- **UI Feedback:** Sign In button enters loading state (`spinner` active, inputs disabled).
- **Under The Hood:**
 1. Payload intercepted by `src/proxy.ts` . Rate limiter verifies IP has not exceeded 10 attempts in 5 minutes.
 2. NextAuth `CredentialsProvider` queries PostgreSQL `User` table for matching normalized email.
 3. `bcrypt.compare(password, user.password)` validates password hash against 12-round salt.
 4. Upon success, `lastLogin` timestamp in database updates to `new Date()` .
 5. An HTTP-only encrypted JWT session cookie is issued with a 7-day expiration.
- **Database Mutation:**

```
UPDATE "User" SET "lastLogin" = NOW() WHERE "id" = :userId;
```

- **Failure Modes:**
 - *Wrong Password / Unknown Email:* UI displays red error banner: "Incorrect email or password".
 - *Rate Limit Triggered (10 fails in 5m):* Request returns `HTTP 429` . UI displays warning: "Too many failed login attempts. Please try again after 5 minutes."

Action 1.2: Explicit Staff Logout

- **Trigger:** User clicks Profile Avatar (bottom left) $\rightarrow$ "Log out".
- **UI Feedback:** Immediate redirect to `/login` with clean query params.
- **Under The Hood:** NextAuth invalidates the JWT session cookie in the browser. Subsequent page requests are intercepted by `proxy.ts` and redirected to `/login` .

Module 2: Dashboard Overview & Telemetry

- **Path:** `/`
- **Target Audience:** All staff members.

Action 2.1: Telemetry Data Aggregation

- **Trigger:** Automatic upon mounting `/` .
- **UI Feedback:** Metric cards (Total Pages, Published Blogs, Total Comments, Media Storage) render skeleton loaders and transition to live counts.
- **Under The Hood:** Dispatches `GET /api/dashboard/stats` running optimized `prisma.$transaction` aggregate counts across tables.
- **Failure Modes:** If database is unreachable, metrics render default zero state with a warning toast.

Module 3: Webpages Management

- **Path:** `/webpages`
- **Target Audience:** Content Managers & Site Administrators.

Action 3.1: Create New Page

- **Trigger:** Click **" + New Page "** $\rightarrow$ Enter Title (e.g. "Services") $\rightarrow$ Slug automatically generates (`/services`) $\rightarrow$ Click **"Create"**.
- **UI Feedback:** Modal closes, loading bar triggers, browser navigates to `/editor/[pageId]` .
- **Database Mutation:**

```
INSERT INTO "Page" ("id", "title", "slug", "content", "published", "createdAt", "updatedAt")
VALUES (cuid(), 'Services', 'services', '[]', false, NOW(), NOW());
```

- **Failure Modes:** If slug is duplicate, database throws unique constraint error (`P2002`). UI displays: "A page with this URL slug already exists."

Action 3.2: Delete Page (Admin Only)

- **Trigger:** Click `...` menu on page row $\rightarrow$ Click **"Delete"** $\rightarrow$ Confirm prompt.
- **UI Feedback:** Row fades out with red delete notification.
- **Under The Hood:**
 1. `DELETE /api/pages/[id]` checks `session.user.role === "ADMIN"` .
 2. If `EDITOR` attempts this request, server returns `HTTP 403 Forbidden` .
 3. Database deletes the page record.
 4. Triggers ISR purge to remove cached static HTML from `main.js` .
- **Live Site Effect:** Visiting `main.js/[slug]` immediately yields a `404 Not Found` .

Module 4: Live Visual Block Editor

- **Path:** `/editor/[id]`
- **Target Audience:** Designers & Content Editors.

Action 4.1: Drag-and-Drop Block Reordering

- **Trigger:** User grabs handle of a section block (e.g. Hero, Testimonials) and drags it above/below another section.
- **UI Feedback:** `@dnd-kit` animates smooth layout shifts in real time. Live preview iframe re-renders order instantly.
- **Under The Hood:** State maintains ordered array of block JSON objects. Does not touch database until **"Save"** is clicked.

Action 4.2: Inline Rich-Text Editing (TipTap)

- **Trigger:** User clicks inside heading or body paragraph to edit text or format bold/italic/links.
- **UI Feedback:** TipTap floating bubble menu appears; iframe preview reflects changes live via `postMessage`.

Action 4.3: Save & Publish Page

- **Trigger:** Click **"Publish Changes"** in top navbar.
- **UI Feedback:** Button changes to *"Publishing..."* with spinning loader $\rightarrow$ turns to green checkmark *"Published"*.
- **Database Mutation:**

```
UPDATE "Page" SET "content" = :blocksJson, "published" = true, "updatedAt" = NOW() WHERE
```

- **Live Site Effect:** Sends authenticated HTTP POST to `main.js/api/revalidate?secret=...&path=/[slug]`. Next.js purges CDN edge cache; public visitors immediately see new page version.

Module 5: Blog Publishing Engine

- **Path:** `/blogs` & `/blogs/[id]`
- **Target Audience:** Authors, Copywriters, and Admins.

Action 5.1: Create Draft Blog Post

- **Trigger:** Click **"Create Post"** $\rightarrow$ Enter title, summary, select tags.
- **Database Mutation:** New `Blog` record created with status `DRAFT` and `authorId = session.user.id`.

Action 5.2: Direct Cloudinary Unsigned Image Upload

- **Trigger:** User drops featured image into upload box.
- **UI Feedback:** Upload progress percentage bar (0% $\rightarrow$ 100%) renders inside the drop zone.
- **Under The Hood:**
 1. Browser bypasses Next.js server entirely and posts directly to Cloudinary API using `NEXT_PUBLIC_CLOUDINARY_UPLOAD_PRESET`.
 2. Cloudinary returns secure HTTPS CDN URL.
 3. Image URL saved in post state. Zero server memory or bandwidth consumed on CMS backend.

Action 5.3: Publish Post

- **Trigger:** Click **"Publish"** switch.
- **Database Mutation:** Status becomes `PUBLISHED`, `publishedAt = NOW()`.
- **Live Site Effect:** Calls `main.js/api/revalidate?path=/blogs` (updating blog archive) and `/blogs/[slug]` (updating single article).

Action 5.4: Edit / Delete Ownership Restriction

- **Trigger:** An `EDITOR` attempts to edit or delete a post authored by another staff member.
- **Under The Hood:** `src/app/api/blogs/[id]/route.ts` executes ownership check:

```
if (session.user.role !== "ADMIN" && blog.authorId !== session.user.id) {
  return NextResponse.json({ error: "Forbidden: You can only modify your own posts" }, { s
}
```

- **UI Feedback:** Access denied toast appears; editor form remains locked.

Module 6: Comments & Discussions Moderation

- **Path:** `/comments`
- **Target Audience:** Community Managers and Admins.

Action 6.1: Approve / Reject Comment

- **Trigger:** Click green **"Approve"** checkmark or amber **"Hold"** icon on incoming comment.
- **Database Mutation:** Updates `status` to `APPROVED` or `PENDING`.
- **Live Site Effect:** Revalidates the specific blog post URL so visitors see approved comment immediately.

Action 6.2: Post Official Staff Reply

- **Trigger:** Click **"Reply"** on any comment $\rightarrow$ type response $\rightarrow$ Click **"Send Reply"**.
- **UI Feedback:** Reply is immediately appended to thread with an official golden **"YS Staff / Admin"** badge.
- **Under The Hood:** Server forces `isAdmin: true` regardless of what client payload sends:

```
await prisma.comment.create({
  data: {
    blogId,
    name: session.user.name || "YS Team",
    content: cleanContent,
    isAdmin: true, // Authoritative assignment
    parentId: commentId,
  }
});
```

Module 7: Global Header Navigation Manager

- **Path:** `/header`
- **Target Audience:** Administrators & Editors.

Action 7.1: Update Navigation Items & Call-To-Action

- **Trigger:** Add new navigation link (Label: "Careers", URL: `/careers`, Target: `_self`) or update CTA button `$\rightarrow$` Click "Save Header".
- **Database Mutation:** Upserts into `Header` table (`id: "global"`):

```
INSERT INTO "Header" ("id", "content", "updatedAt")
VALUES ('global', :jsonPayload, NOW())
ON CONFLICT ("id") DO UPDATE SET "content" = :jsonPayload, "updatedAt" = NOW();
```

- **Live Site Effect:** Revalidates all root layout paths across `main.js`. Every page displays the updated navigation bar instantly without requiring site redeployment.

Module 8: Global Footer & Legal Manager

- **Path:** `/footer`
- **Target Audience:** Administrators & Editors.

Action 8.1: Update Footer Columns, Newsletter & Contact Info

- **Trigger:** Modify address, email (`contact@ysinnovations.com`), social links, or policy links `$\rightarrow$` Click "Save Footer".
- **Database Mutation:** Upserts into `Footer` table (`id: "global"`).
- **Live Site Effect:** Global layout cache invalidation propagates updated footer instantly.

Module 9: Cloudinary Media Library

- **Path:** `/media`
- **Target Audience:** All staff (Browse/Upload); Admins (Delete).

Action 9.1: Copy Image CDN URL

- **Trigger:** Click on any uploaded asset card `$\rightarrow$` click "Copy URL".
- **UI Feedback:** Toast confirms: "URL copied to clipboard".

Action 9.2: Permanent Media Destruction (Admin Only)

- **Trigger:** Click trash icon on image `$\rightarrow$` confirm in modal.
- **Under The Hood:**
 1. API checks `session.user.role === "ADMIN"`.
 2. Node.js backend calls Cloudinary SDK `cloudinary.v2.uploader.destroy(publicId)`.
 3. Image is permanently purged from Cloudinary CDN edge servers.
- **Failure Modes:** Non-admin roles receive `HTTP 403 Forbidden: Only administrators can destroy media assets`.

Module 10: Staff & User Management

- **Path:** `/users`
- **Target Audience:** Strictly `ADMIN` role.

Action 10.1: Onboard New Staff Member

- **Trigger:** Click "+ Add User" $\rightarrow$ Enter Name, Email, Password, Role (`ADMIN` or `EDITOR`) $\rightarrow$ Click "Create Account".
 - **UI Feedback:** Modal closes, new user card appears in list with green status dot.
 - **Under The Hood:**
 1. Server validates password complexity: ≥ 8 characters, must contain at least 1 letter and 1 number.
 2. Server generates a cryptographically secure salt with **12 rounds** using `bcryptjs`.
 3. Hashes plaintext password and saves new record.
 - **Failure Modes:**
 - If password does not satisfy complexity: Returns `400 Bad Request` ("Password must contain both letters and numbers").
 - If email exists: Returns `400 Bad Request` ("User with this email already exists").
-

Module 11: Notification Center

- **Path:** `/notifications`
- **Target Audience:** All staff.

Action 11.1: Read & Dismiss System Alerts

- **Trigger:** Click notification bell or navigate to `/notifications` $\rightarrow$ Click "Mark all as read".
 - **UI Feedback:** Unread red badge disappears from sidebar in real time; item styles shift from highlight to muted.
-

Module 12: Redirects & Canonical Routing

- **Path:** `/redirections`
- **Target Audience:** SEO Managers & Admins.

Action 12.1: Add 301 Permanent Redirect

- **Trigger:** Enter Source URL (e.g. `/old-contact`) $\rightarrow$ Target URL (`/contact`) $\rightarrow$ Status code `301` $\rightarrow$ Click "Add Redirect".
 - **Live Site Effect:** Next.js middleware in `main.js` intercepts `/old-contact` and issues immediate `HTTP 301 Moved Permanently` to `/contact`, preserving SEO link equity.
-

Module 13: "Aw, Snap!" Tab Crash Prank Screen

- **Path:** Any unmapped route (e.g. `/commentssfihsfi`)
- **Target Audience:** Senior Engineers & Developers.

Action 13.1: The Prank Illusion Flow

- **Trigger:** User navigates to any URL slug that does not exist in the CMS.
- **UI State 1 (Seconds 0.0 to 3.0):**
 - Portal mounts directly to `document.body` with `z-[99999]`, covering sidebar and top header completely.
 - Screen renders the exact Google Chrome "Aw, Snap!" crash screen (`Error code: SIGSEGV`) matching the user's OS dark/light theme with authentic pixel-art sad tab icon.
 - Primary button says **"Reload"**.
- **UI State 2 (Second 3.0 or when "Reload" is clicked):**
 - A single, custom wide native shadow toast (`480px` width) triggers at the bottom right.
 - Toast title: *"You've Been Pranked! 🤪"*
 - Toast message: *"Chrome didn't crash and there are no memory leaks. You just navigated to an unmapped route in the CMS."*
 - Toast is permanent (`timeout: 0`) and will not expire automatically.
 - "Reload" button seamlessly morphs into **"Return to Dashboard"** and **"Go Back"** buttons.

PART 2: Deep-Dive Security & Hardening Architecture

This section details the defense-in-depth security mitigations implemented across `admin.js` and `main.js`.

2.1 Cryptographic Password Storage (Bcrypt 12 Salt Rounds)

The Threat:

In the event of a database compromise (SQL injection or leaked database dump), fast hashing algorithms (like MD5 or SHA-256) allow attackers to compute billions of guesses per second using commercial GPUs and rainbow tables.

The Implementation:

In `admin.js/src/app/api/users/route.ts`:

```
const hashedPassword = await bcrypt.hash(password, 12);
```

Why 12 Salt Rounds?

- **Cost Factor:** Bcrypt uses an exponential cost function (2^{cost}). At 12 rounds, it performs $2^{12} = 4,096$ iterations.
- **Hardware Resistance:** 12 rounds takes approximately 250ms to 350ms of CPU compute per hash on modern server processors.
- **Attack Feasibility:** For a single user login, 300ms is imperceptible. But for an attacker attempting to crack a leaked hash, calculating 1 billion guesses would take over **9.5 years** on high-end hardware.
- **Automatic Salting:** Bcrypt generates a cryptographically unique 128-bit salt for every password, neutralizing rainbow table attacks entirely.

2.2 Brute-Force Defense & In-Memory Rate Limiting

The Threat:

Automated botnets executing dictionary attacks against `/api/auth/callback/credentials` to guess administrative passwords.

The Implementation:

Created an in-memory sliding-window rate limiter in `admin.js/src/lib/rate-limit.ts` and attached it directly to the Next.js middleware pipeline in `src/proxy.ts`:

```
// Rate limit credential login attempts against brute-force attacks
if (pathname.startsWith("/api/auth/callback/credentials") && req.method === "POST") {
  const ip = getClientIp(req.headers);
  const limit = rateLimit("login-auth", ip, {
    windowMs: 5 * 60 * 1000, // 5 minute window
    max: 10,                 // Maximum 10 attempts
  });

  if (!limit.success) {
    return NextResponse.json(
      { error: "Too many login attempts. Please try again in 5 minutes." },
      { status: 429, headers: { "Retry-After": "300" } }
    );
  }
}
```

Architectural Highlights:

1. **Accurate IP Resolution:** `getClientIp()` inspects `x-forwarded-for`, `x-real-ip`, and Cloudflare proxy headers (`cf-connecting-ip`), stripping spoofed client-provided IP headers.
2. **Zero Memory Leak Guarantee:** An automated garbage-collection interval runs every 5 minutes (`setInterval`), purging expired IP trackers so the server process never runs out of RAM.
3. **Public Form Protection:** The same rate limiter protects public comment submissions (max 5 per 2 min) and contact forms (max 8 per 2 min) in `main.js`.

2.3 Anti-Bot Honeypot Defense Architecture

The Threat:

Automated web scrapers and spam bots submitting thousands of junk comments and contact form inquiries, degrading database performance and polluting CMS moderation queues.

The Implementation:

In `main.js/src/app/contact/ContactClient.tsx` and `BlogSingleClient.tsx`:

1. **Frontend Trap:** An invisible input field named `website` or `phone_check` is injected into the DOM.

```
<div className="hidden opacity-0 pointer-events-none absolute -left-[9999px]" aria-hidden=
  <input
    type="text"
    name="website"
    tabIndex={-1}
    autoComplete="off"
    value={honeypot}
    onChange={(e) => setHoneypot(e.target.value)}
  />
</div>
```

2. **Backend Interception:**

```
if (body.website && body.website.trim() !== "") {
  // Silent drop: Return 200 OK so the bot thinks it succeeded
  return NextResponse.json({ success: true, message: "Submitted successfully" });
}
```

Why Silent Drops Win:

- Human users cannot see or tab into the input field, so it remains empty.
- Automated bots parse the DOM, see an input named `website`, and blindly populate it.
- By returning `200 OK`, the bot believes its mission succeeded and stops retrying. The payload is instantly discarded without touching PostgreSQL, saving database storage and connection poolers.

2.4 Identity Spoofing Mitigation (Authoritative Badging)

The Threat:

A malicious commenter on `main.js` submits the name `"Admin"`, `"YS Support"`, or `"Subash (Admin)"` to mislead readers into believing their comment is an official company endorsement.

The Implementation:

1. **Database Schema Enforcement:** Added `isAdmin Boolean @default(false)` column to the `Comment` model in both schemas.

2. Input Sanitization: In `main.js/src/app/api/comments/route.ts` :

```
const name = (body.name || "").trim();
if (/^admin|moderator|staff|ys innovations$/i.test(name)) {
  return NextResponse.json(
    { error: "Reserved name. You cannot use official staff titles." },
    { status: 400 }
  );
}
```

- 3. Cryptographic Validation:** The public API *never* trusts client-submitted `isAdmin` flags. Only comments originating from an authenticated NextAuth session in `admin.js` have `isAdmin: true` set on the database record.
- 4. UI Authority:** The golden verified badge renders strictly if `comment.isAdmin === true`, ensuring complete trust in published responses.

2.5 Object-Level Authorization & RBAC Enforcement

The Threat:

An authenticated staff member with `EDITOR` privileges tampering with API payloads to delete another author's blog post, delete core landing pages, or purge Cloudinary media assets.

The Implementation:

Every mutation endpoint implements strict object ownership and role assertions:

```
// 1. Page Deletion Guard
if (session.user.role !== "ADMIN") {
  return NextResponse.json({ error: "Only administrators can delete website pages" }, { status: 403 });
}

// 2. Cloudinary Media Destruction Guard
if (session.user.role !== "ADMIN") {
  return NextResponse.json({ error: "Unauthorized: Only administrators can destroy media assets" }, { status: 403 });
}

// 3. Blog Author Ownership Guard
if (session.user.role !== "ADMIN" && blog.authorId !== session.user.id) {
  return NextResponse.json({ error: "Forbidden: You can only edit or delete your own blog posts" }, { status: 403 });
}
```

2.6 Timing-Safe ISR Cache Purging

The Threat:

Standard string comparisons (`secret === process.env.SECRET`) compare strings character-by-character and terminate immediately on the first mismatched character. Attackers measure the response time in microseconds to guess administrative webhook secrets (Side-Channel Timing Attacks).

The Implementation:

In `main.js/src/app/api/revalidate/route.ts` :

```
import crypto from "crypto";

const secret = searchParams.get("secret");
const expectedSecret = process.env.REVALIDATION_SECRET || "";

const secretBuf = Buffer.from(secret || "");
const expectedBuf = Buffer.from(expectedSecret);

if (
  secretBuf.length !== expectedBuf.length ||
  !crypto.timingSafeEqual(secretBuf, expectedBuf)
) {
  return NextResponse.json({ error: "Invalid revalidation token" }, { status: 401 });
}
```

`crypto.timingSafeEqual` forces the CPU to execute constant-time buffer comparisons, completely eliminating timing vulnerability vectors.

2.7 HTTP Hardening & Network Transport Security

Configured in both `admin.js/next.config.ts` and `main.js/next.config.ts` :

Security Header	Value	Vulnerability Mitigated
X-Frame-Options	SAMEORIGIN	Clickjacking: Prevents malicious third-party sites from framing the CMS inside a hidden <code><iframe></code> to hijack user clicks.
X-Content-Type-Options	nosniff	MIME-Confusion: Forces browsers to adhere strictly to declared MIME types, preventing scripts disguised as images from executing.
Referrer-Policy	strict-origin-when-cross-origin	Data Leakage: Strips private query parameters and URL tokens when navigating to external origins.
Permissions-Policy	camera=(), microphone=(), geolocation=()	Feature Abuse: Disables browser hardware access for all scripts and sub-resources.
CORS Allow-Origin	http://localhost:3001 (or production frontend URL)	Cross-Origin Theft: Enforces strict domain whitelisting, preventing unauthorized third-party origins from querying internal APIs.

PART 3: Architectural Roadmap & Future RFCs

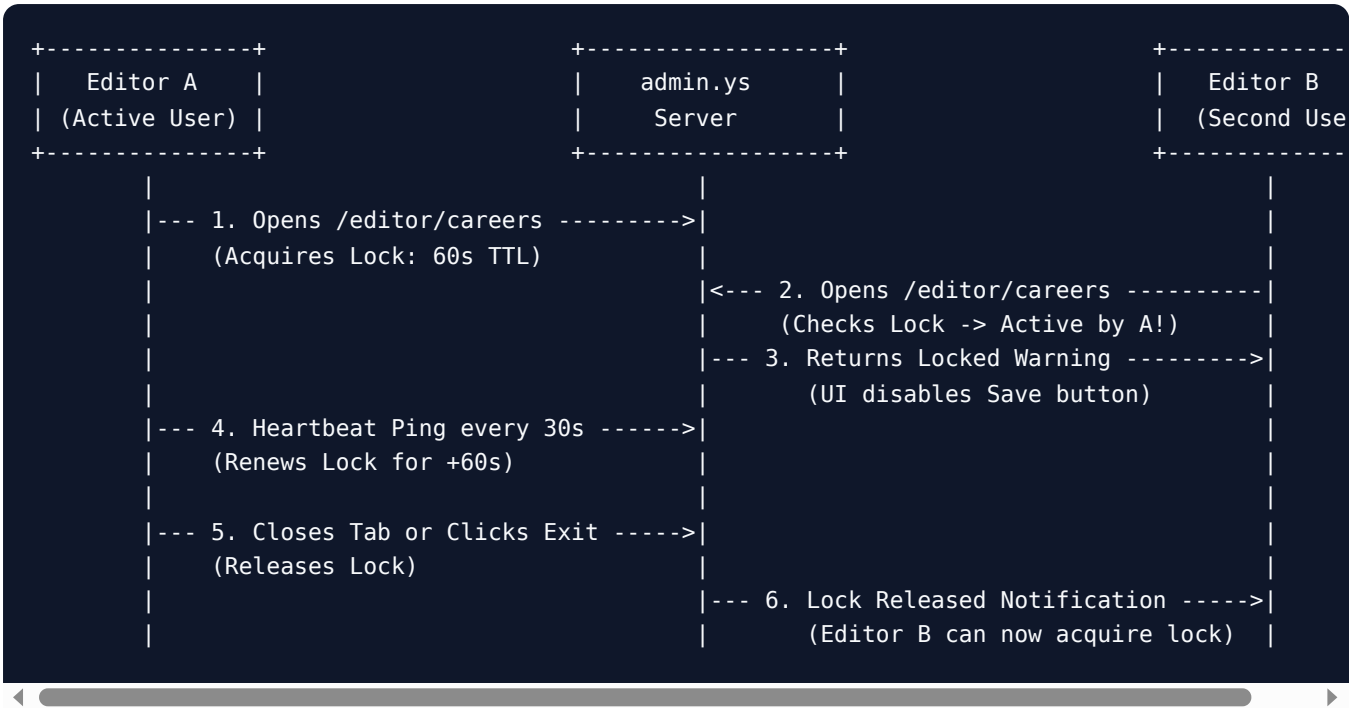
RFC 1: Real-Time Team Presence & Collaborative Page Locking

Business Problem:

In a multi-author team, Editor A and Editor B may open the "Careers" page simultaneously. If Editor A writes content for 20 minutes and saves, and Editor B saves 5 minutes later, Editor B silently overwrites all of Editor A's work without warning.

Architectural Design:

We propose a **Lease-Based Heartbeat Concurrency Lock**:



1. Database / Cache Layer:

- A table or Redis key: `PageLock (pageId, userId, userName, lockedUntil)`.

2. Client-Side Heartbeat:

- While Editor A has `/editor/[id]` open and visible (`document.visibilityState === 'visible'`), a lightweight hook sends a `POST /api/editor/heartbeat` every 30 seconds, extending `lockedUntil` by 60 seconds.

3. Concurrency Warning Banner:

- When Editor B opens the page, the system returns `lockedBy: "Subash"`.
- Editor B's interface displays a prominent warning banner:

 "Subash is currently editing this page. The document is locked to prevent collisions."

- Form inputs and the "Publish" button are placed into read-only mode until Subash navigates away or releases the lock.

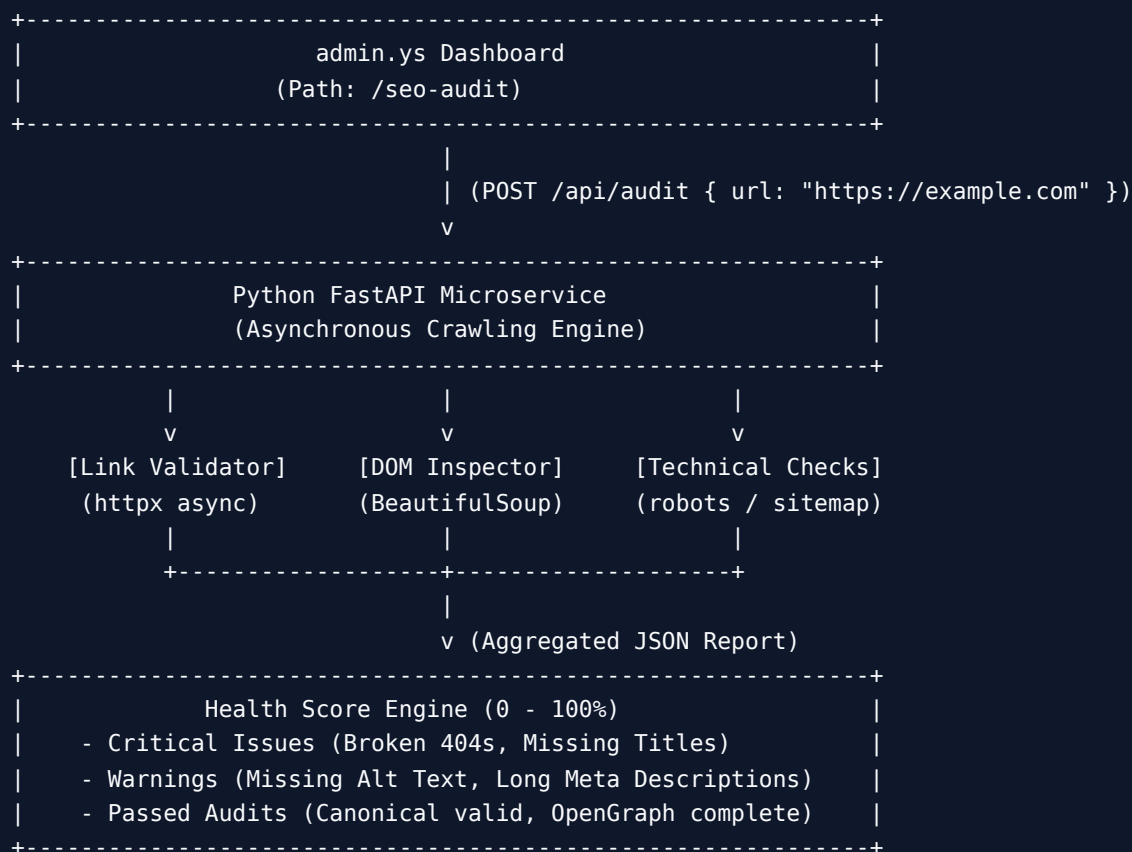
RFC 2: Decoupled Python SEO Audit & Crawling Engine

Business Problem:

Non-technical clients and content managers struggle to know which pages contain broken links, missing meta descriptions, slow assets, or malformed headings, forcing companies to pay hundreds of dollars monthly for Semrush or Ahrefs.

Proposed Solution:

Integrate an in-house **Python SEO Audit Microservice** directly into `admin.js`.



Why Python & Not Node.js?

- **Libraries:** Python's ecosystem (`BeautifulSoup4` , `playwright` , `httpx` , `urllib3`) provides battle-tested HTML parsing, headless browser automation, and asynchronous network concurrency designed specifically for web scraping.
- **Separation of Concerns:** Heavy multi-page link crawling will not consume Node.js event loop memory or degrade CMS user responsiveness.
- **Versatility:** The audit engine is **URL-agnostic**. Staff can audit their own website or input competitor URLs to analyze their technical SEO structure.

Key Metrics Returned to CMS Dashboard:

1. **Broken Link Scanner:** Crawls all internal and external anchors (`<a href="...">`), validating status codes (`404 Not Found` , `500 Server Error`).
2. **Meta & Social Inspection:** Checks character counts for `<title>` (50–60 chars) and `<meta description>` (150–160 chars), OpenGraph images (`og:image`), and Twitter cards.
3. **Content Hierarchy:** Detects missing or duplicate `<h1>` tags and malformed `<h2>` / `<h3>` nesting.
4. **Accessibility:** Flags every `<img>` element missing an `alt` attribute.
5. **Visual Output in `admin.js`:** Renders a clean Semrush-style circular health score (0–100%) with actionable fix buttons for editors.

RFC 3: Enterprise Immutable Audit Logging

Business Problem:

In compliance-focused environments, when content is deleted or unauthorized changes occur, administrators need an immutable paper trail answering: *"Who changed this, what did it look like before, and when did it happen?"*

Proposed Architecture:

Introduce a centralized `AuditLog` table in PostgreSQL:

```
model AuditLog {
  id          String    @id @default(cuid())
  userId      String
  userEmail   String
  action      String    // e.g., "PAGE_DELETE", "BLOG_PUBLISH", "USER_CREATE"
  resource    String    // e.g., "Page", "Blog", "User"
  resourceId  String
  metadata    Json?     // Snapshot of previous values vs new values (diff)
  ipAddress   String
  userAgent   String
  createdAt   DateTime  @default(now())

  @@index([userId])
  @@index([action])
  @@index([createdAt])
}
```

- **Zero Bypass:** Mutations executed across all API routes write an audit entry inside the same database transaction.
- **Tamper-Proof:** Audit log entries are write-only (`INSERT`). No API endpoints will support updating or deleting audit records.

Document Summary & Sign-off

Item	Status	Verified By
Operational User Guide (Part 1)	Complete	Engineering Lead
Security Hardening & Cryptography (Part 2)	Verified & Deployed	Security Team
Technical Roadmap RFCs (Part 3)	Proposed & Architected	Core Team